

Prompt

Write a script for reconciled set of books: know exactly which amounts are missing or unaccounted for, and who you still need to follow up with

Expected Total

- Expected Counted Total: PKR 42,000
- Context: This is the pool collected for a cricket screening and biryani night for the apartment block. Everyone was supposed to chip in exactly PKR 3,500 per household. Based on the physical headcount/RSVPs, 12 apartments confirmed they paid.

Flat #	Resident / Family Name
101	Asad Khan
102	Tariq Mahmood
201	Zainab Bibi
204	The Chishti Family
302	Bilal Ahmed
305	Ayesha Omar
401	Kamran M.
402	Asif Shah
406	Salman Raza
501	Haris Abbasi
502	Sana Niazi
601	The Jafri Family

2. Messy Digital Payment Record

Here is the raw, unedited export from the digital payment app:

[TIMESTAMP: 11/07] TXN_ID: 44102 AMT: 3,500 PKR MEMO: "Biryani" FROM: a_khan92

[TIMESTAMP: 11/07] TXN_ID: 44103 AMT: 3,500 PKR MEMO: "match night" FROM: Zainab Bibi

[TIMESTAMP: 11/07] TXN_ID: 44104 AMT: 3,500 PKR MEMO: "🔪🔥" FROM: Bilal Ahmed

[TIMESTAMP: 11/07] TXN_ID: 44105 AMT: 7,000 PKR MEMO: "Apt 401 and 402" FROM: Kamran M.

[TIMESTAMP: 11/07] TXN_ID: 44106 AMT: 3,500 PKR MEMO: "screen rent" FROM: Flat 10-C

[TIMESTAMP: 11/07] TXN_ID: 44107 AMT: 3,000 PKR MEMO: "baki kal donga"
FROM: Tariq Mahmood

[TIMESTAMP: 11/07] TXN_ID: 44108 AMT: 3,500 PKR MEMO: "easypaisa transfer"
FROM: Apex Tech Solutions

[TIMESTAMP: 11/07] TXN_ID: 44109 AMT: 3,500 PKR MEMO: "S. N." FROM:
user_9921_pk

[TIMESTAMP: 11/07] TXN_ID: 44110 AMT: 3,500 PKR MEMO: "Match pool"
FROM: The Jafri Family

Personal Interpretation Rules

To map these ambiguous entries to the correct residents, use these rules:

- "Kamran M." paid PKR 7,000 to cover his own flat (401) and his neighbor Asif Shah (402).
- "Flat 10-C" is rented by Ayesha Omar.
- "Apex Tech Solutions" is the business account used by Salman Raza, which he mistakenly used instead of his personal account.
- "user_9921_pk" with the memo "S. N." belongs to Sana Niazi.

Logic

Data Structuring (The Input State)

Before processing anything, the script sets up two core data pools:

- **The Household Ledger (Master List):** An array of dictionaries containing the source of truth—who lives in which flat, and the expected amount (PKR 3,500) they owe.
- **The Raw Transactions:** A list of the unedited records pulled from the digital payment app, containing ambiguous names (a_khan92, Flat 10-C), varying amounts, and custom notes.

2. The In-Memory Tracker

To keep track of money as it gets processed, the script initializes a tracking dictionary using a **Dictionary Comprehension**:

Python

```
payments_by_flat = {h["flat"]: 0 for h in households}
```

This instantly builds a dynamic scoreboard where every single flat number starts with a balance of 0. As the script reads payments, it adds money directly to these flat keys.

3. The Mapping & Translation Engine

The script runs a loop over every transaction in the raw app dump. Inside this loop, it uses an if-elif chain (conditional logic) acting as a custom translation matrix based on your interpretation rules:

- **Direct String Matching:** Matches explicit names like Zainab Bibi or Bilal Ahmed directly to their respective flats.
- **Aliasing & Context Rules:**
 - If the sender is Flat 10-C, the script routes the money to Ayesha Omar's flat (305).
 - If the sender is Apex Tech Solutions, it routes the money to Salman Raza's flat (406).
- **Splitting Transactions (Mathematical Logic):** When it encounters Kamran M. sending PKR 7,000, the script divides the sum, adding PKR 3,500 to Flat 401 and PKR 3,500 to Flat 402 simultaneously.
- **HTML Row Compilation:** As each entry is matched, the script immediately wraps the clean variables inside HTML `<tr>` (table row) and `<td>` (table data) elements, building the **Audit Trail** string piece by piece.

4. Variance & Financial Calculation

Once all payments are mapped to their respective flats, the script runs a second loop through the master household list to calculate the final ledger state:

`$$Variance = Paid - Expected$$`

Using this formula, the script evaluates the results:

- **\$Variance = 0\$:** The household paid exactly what was expected. The script assigns a green FULLY PAID badge.
- **\$Variance < 0\$ and \$Paid > 0\$:** The resident paid something, but fell short (e.g., Tariq Mahmood paying 3,000 instead of 3,500). A yellow PARTIAL badge is assigned, and a warning is appended to the **Follow-Ups** box.
- **\$Paid = 0\$:** No payment was recorded for this flat. A red UNPAID badge is generated, and a critical missing notice is sent to the **Follow-Ups** box.

5. HTML Document Generation (The Output State)

Finally, the script uses an **f-string (formatted string literal)** template. It injects the computed totals (Expected, Collected, Deficit), drops the pre-compiled HTML table rows right into the middle of the layouts, and attaches the CSS styles.

The standard Python file handling command (with `open()` as `f: f.write()`) drops this completed string onto your hard drive as a fully operational `.html` file, ready to render in any browser instantly.